INPUT, TYPE CONVERSION, AND RAW VALUES

Turning typed text into useful data

```
quantity_text = input("Enter quantity: ")
quantity = int(quantity_text)

total = quantity * 25
print(total)
```

Input starts as text. Programs decide what it means.

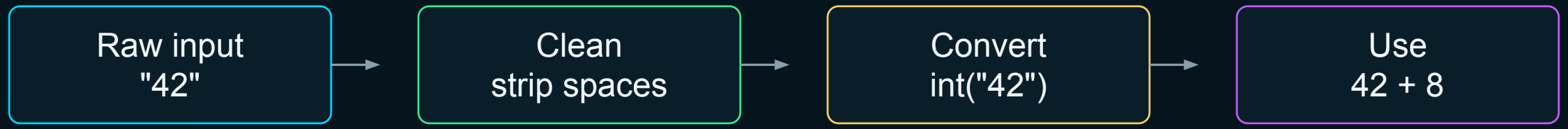
```
status_input = input("Status: ")
status = status_input.strip().lower()

is_active = status == "active"
```

30 min instruction • 30 min guided lab • interactive project calculator

INSTRUCTION

The big idea: raw values need interpretation

Programs receive values, clean them, convert them, and then use them.



Python can read almost anything as text. Your code decides when text becomes a number, Boolean, or cleaned string.

INSTRUCTION

Hard-coded values vs user input

Hard-coded values are written into the program. Input values arrive while the program runs.

```
# Hard-coded values
completed_tasks = 18
total_tasks = 24

percent = completed_tasks / total_tasks * 100
```

```
# User input
completed_text = input("Completed tasks: ")
total_text = input("Total tasks: ")

# convert before calculating
```

Hard-coded: predictable, simple, not flexible

Input: flexible, but needs checking and conversion

INSTRUCTION

Reading input with input()

The input() function pauses the program and waits for the user to type something.

```
employee_name = input("Enter employee name: ")
```

The prompt is not the value. It is just the message asking the user what to enter.

INSTRUCTION

Even numbers arrive as strings

If the user types 42, input() returns the string "42" first.

```
age = input("Age: ")

print(age)
print(type(age))
```

User typed: 42

Python has: "42"

Type: str

Quotation marks are not typed by the user. They show Python is treating the value as text.

INSTRUCTION

Two steps: receive text, then convert

For beginners, the longer version makes the process visible.

```
quantity_text = input("Enter quantity: ")
quantity = int(quantity_text)
```

Step 1: receive text

Step 2: convert text to int

```
quantity = int(input("Enter quantity:
"))
```

Shorter version

Same idea, less explicit

INSTRUCTION

Why conversion matters

Without conversion, + joins strings instead of adding numbers.

```
first_value = input("Enter a number: ")
second_value = input("Enter another number:
")

result = first_value + second_value

# User enters 10 and 5
```

Result: "105"

```
first_value = int(input("Enter a number: "))
second_value = int(input("Enter another number: "))

result = first_value + second_value
```

Result: 15

INSTRUCTION

Common conversion functions

Conversion tells Python how to interpret a raw value.

`int()`	whole number	`int("1250")`	`1250`
`float()`	decimal number	`float("72.5")`	`72.5`
`str()`	text representation	`str(404)`	`"404"`

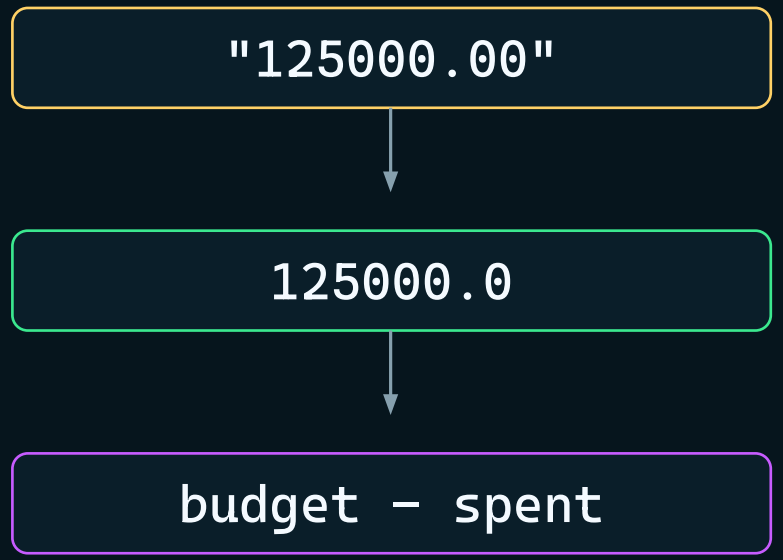
Conversion does not magically fix bad input. It only works when the text is compatible with the target type.

INSTRUCTION

Convert before calculating

Calculations should usually use numeric values, not raw input strings.

```
budget_text = input("Budget: ")
spent_text = input("Amount spent: ")

budget = float(budget_text)
amount_spent = float(spent_text)

remaining_budget = budget - amount_spent
```



Beginner rule: convert at the edge of the program, then use clean variables afterward.

INSTRUCTION

Conversion failures

Conversion requires compatible text. Some inputs cannot become numbers.

```
record_count = int("1250")   # works
temperature = float("72.5")  # works

record_count = int("twelve") # fails
```

ValueError

Python cannot safely guess what “twelve” should become.

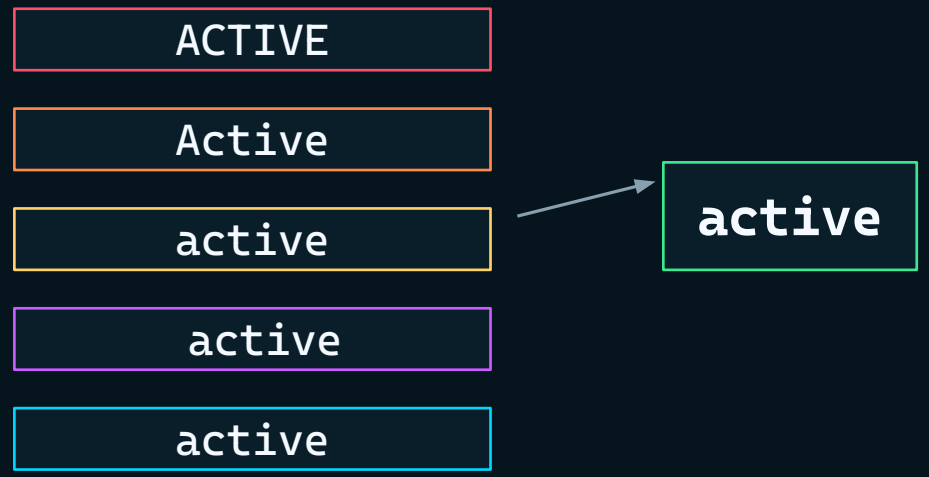
For now: understand the failure.
Later: use validation and exception handling to recover gracefully.

INSTRUCTION

Normalize before comparing

Raw text often includes inconsistent spaces or capitalization.

```
status_input = input("Enter status: ")
status = status_input.strip().lower()

is_active = status == "active"

print(f"Active record: {is_active}")
```



Clean strings before comparing them to expected values.

INSTRUCTION

Raw values versus interpreted values

The same text can be interpreted differently depending on what the program needs.

```
raw_value = "404"
```

as text
"404" — part of a filename

as number
404 — status/error code

as display
"Error 404" — human message

as category
not found — application meaning

The raw characters are only the starting point. Software gives them meaning.

INSTRUCTION

Why files and networks often arrive as text

Data often crosses system boundaries in plain text formats.

CSV file
"EQ-1045,active,820"

Web form
"budget=125000"

API response
"temperature": "72.5"

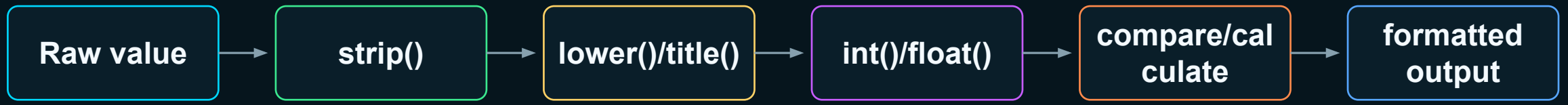
Log line
"ERROR,404,/status"

Real programs constantly parse raw text from files, websites, APIs, forms, logs, and sensors.

INSTRUCTION

Basic data-parsing workflow

A repeatable pattern for turning messy input into useful data.




```
status_input = input("Enter status: ")
status = status_input.strip().lower()
is_active = status == "active"

print(f"Active record: {is_active}")
```

DEMONSTRATION

Demo: interactive project calculator

Ask for raw values, then convert only the values that need numeric math.

```
project_name = input("Project name: ").strip()
completed_tasks = int(input("Completed tasks: "))
total_tasks = int(input("Total tasks: "))
budget = float(input("Budget: "))
amount_spent = float(input("Amount spent: "))
approval_text = input("Approved? yes/no:
").strip().lower()
```

Text kept as text

```
project_name
approval_text
```

Text converted to numbers

```
completed_tasks
total_tasks
budget
amount_spent
```

DEMONSTRATION

Calculate after conversion

Once the values are converted, the math is normal Python math.

```
completion_percentage = completed_tasks / total_tasks
* 100
remaining_budget = budget - amount_spent
is_approved = approval_text == "yes"
```

tasks / tasks
→ percentage

budget - spent
→ remaining

"yes" check
→ Boolean

Notice the pattern: input, conversion, calculation, comparison, output.

DEMONSTRATION

Print a formatted project summary

Formatted output turns internal values into a report a human can read.

```
print(f"Project: {project_name}")
print(f"Completed: {completed_tasks} of
{total_tasks}")
print(f"Completion:
{completion_percentage:.1f}%")
print(f"Budget remaining:
${remaining_budget:.2f}")
print(f"Approved: {is_approved}")
```

```
Project: Sensor Upgrade
Completed: 18 of 24
Completion: 75.0%
Budget remaining: $31250.00
Approved: True
```

The output is not how the values are stored internally. It is a human-readable interpretation.

GUIDED LAB

Guided exercise: build the project calculator

Students create an interactive program that asks, converts, calculates, and prints.

- Project name
- Completed tasks
- Total tasks
- Budget
- Amount spent
- Approved? yes/no

Lab goal: track which inputs stay strings and which ones become int, float, or Boolean values.

Students should run the program several times with different values.

GUIDED LAB

Lab starter: input and conversion

Type the input lines first. Test before adding calculations.

```
project_name = input("Project name: ").strip()
completed_tasks = int(input("Completed tasks: "))
total_tasks = int(input("Total tasks: "))
budget = float(input("Budget: "))
amount_spent = float(input("Amount spent: "))
approval_text = input("Approved? yes/no:
").strip().lower()
```

```
print(type(project_name))
print(type(completed_tasks))
print(type(total_tasks))
print(type(budget))
print(type(amount_spent))
print(type(approval_text))
```

Checkpoint: before doing the math, make sure the types are what you expect.

GUIDED LAB

Lab next step: calculations and Boolean interpretation

Convert yes/no text into a Boolean expression.

```
completion_percentage = completed_tasks / total_tasks * 100
remaining_budget = budget - amount_spent
is_approved = approval_text == "yes"
```

completion_percentage
float

remaining_budget
float

is_approved
bool

The Boolean is not typed by the user. It is created by comparing normalized text to "yes".

GUIDED LAB

Lab final: formatted summary

Readable output makes the program feel like a real tool.

```
print(f"Project: {project_name}")
print(f"Completed tasks: {completed_tasks} of
{total_tasks}")
print(f"Completion: {completion_percentage:.1f}%")
print(f"Budget: ${budget:.2f}")
print(f"Spent: ${amount_spent:.2f}")
print(f"Remaining: ${remaining_budget:.2f}")
print(f"Approved: {is_approved}")
```

```
Project: Sensor Upgrade
Completed tasks: 18 of 24
Completion: 75.0%
Budget: $125000.00
Spent: $93750.00
Remaining: $31250.00
Approved: True
```

Predict the output before running it, then compare your prediction to the actual result.

GUIDED LAB

What if the user enters something unexpected?

Students identify failures and weird-but-legal inputs.

`Completed tasks: eighteen`	int conversion fails
`Budget: 125,000`	float conversion fails
`Approved? y`	is_approved becomes False
`Project name:`	empty-ish project name
`Total tasks: 0`	division by zero later

Today: recognize the issue. Later: validate input and handle errors cleanly.

DISCUSSION

Exercise discussion: trace the data types

Students explain what each value was before and after processing.

Began as text
every input() result

Converted to int
completed_tasks, total_tasks

Converted to float
budget, amount_spent

Converted to Boolean idea
is_approved comparison

Could fail
bad numeric text

Data type questions are not trivia. They explain what operations are safe.

APPLIED CHALLENGE

Applied mini-challenge: raw-to-useful parser

Students make a small input program that cleans, converts, and reports.

Ask for:
- File name
- Size in bytes
- Status
- Records processed
- Total records

Then produce:
- Normalized file name
- Size in MB
- Completion percent
- Active status Boolean
- Human-readable summary

Challenge rule: no calculations using raw input strings.

REVIEW

Knowledge check: input and interpretation

Students should be able to explain these in plain language.

What does input() return?

Why did 10 + 5 become 105?

When should you use int() or float()?

Why normalize before comparing?

What can cause conversion to fail?

What is the raw-to-useful data workflow?

Next step: use input-driven Booleans to choose what code runs with if statements.